

Lesson 2 — Minimal MCP with FastMCP

Goal: understand why MCP exists and build a small custom MCP server that Lesson 3 can connect to the order assistant from Lesson 1.

We will cover only what is needed:

1. Tool vs MCP
2. Host, client, server, and the three MCP primitives
3. A typed order-status MCP server
4. Tool discovery and invocation
5. The exact bridge to Lesson 3

This lesson does not call an LLM, so it needs no API key or `.env` file.

Before we write code: what is MCP?

An AI model can generate text, but it cannot automatically read your database, inspect an order, or call an internal service. To do those things, an application gives the model **tools**: named capabilities with descriptions and structured inputs.

A direct tool works well when the function and the AI application live together. The application imports the function, converts its arguments into a schema, sends that schema to the model, executes the function when requested, and returns the result to the model.

The difficulty appears when the same capability must be used by several AI applications. Each application may use a different framework and may need its own integration code. Authentication, connection handling, schemas, and errors are repeatedly implemented.

Model Context Protocol (MCP) is an open protocol that standardizes this connection. An MCP server describes the capabilities it exposes, while an MCP client can discover and use them through a common interface. The server and client do not need to use the same programming language or AI framework.

A useful analogy is **USB-C for AI capabilities**: USB-C does not create your keyboard or storage device; it provides a shared way to connect them. Similarly, MCP does not create the order lookup—it provides a shared way for AI applications to discover and call it.

The four pieces in plain language

Piece	Meaning in this course
Model	Decides whether it needs order information and supplies arguments such as <code>A100</code>
Host	The AI application the user interacts with; in Lesson 3 this is our LangChain application
MCP client	Lives inside the host, connects to a server, discovers capabilities, and sends requests
MCP server	Publishes capabilities and executes them; here it owns <code>get_order_status</code>

The model does **not** connect directly to the MCP server. The host controls the connection, decides which capabilities are available, executes approved calls, and returns results to the model. This separation is important because application code remains responsible for permissions, validation, logging, and user approval.

What happens during one request?

Suppose the user asks: **What is the status of order A100?**

1. The host connects its MCP client to the order MCP server.
2. The client asks the server which tools are available. This is **discovery**.
3. The server describes `get_order_status`, including its purpose and required `order_id` input.
4. The host makes that tool description available to the model.
5. The model requests `get_order_status(order_id="A100")`; it does not execute the function itself.
6. The host sends the request through the MCP client to the server.
7. The server runs trusted application code and returns structured data.
8. The host gives the result to the model, which writes the final user-facing answer.

User question

↓

Host + model — chooses a discovered tool

↓

MCP client — protocol request → MCP server → order data

↑

└──────── structured result ─────────┘

MCP is the communication layer—not the model, not an agent, and not the business logic. You can use MCP without an LLM, which is exactly how we test the server in this lesson.

1. Why MCP?

In Lesson 1, `get_order_status` was a Python function registered directly with one LangChain application:

LangChain application → Python function

That is ideal when one application owns everything. But every additional AI host would need its own wrapper, schema conversion, connection logic, and error handling.

MCP places a standard boundary around capabilities:

AI host → MCP client → standard protocol → MCP server → order system

The server publishes what it can do. Compatible hosts can discover and call it without importing its Python function.

Tool vs MCP

Question	Direct tool	MCP server
What is it?	A function wired into one app	A standard way to expose tools and context
Discovery	App registers it manually	Client asks the server what it exposes
Boundary	Usually in the same codebase	In-process, local process, or remote service
Reuse	Framework-specific adapter	Reusable by compatible hosts
Best fit	One small application	Shared integrations and independent services

Important: MCP does not replace tools. A tool is a capability; MCP standardizes how a server advertises and executes that capability.

2. MCP capabilities: tools, resources, and prompts

User

↓

Host application (Lesson 3: LangChain + OpenAI)

↓ owns

MCP client

↓ connects to

MCP server (this lesson: order capabilities)

↓ reads

Order data

An MCP server can expose three complementary kinds of capability. They travel through the same protocol but solve different problems.

Primitive	What it provides	Usually selected by	Use it when	Order example
Tool	An executable operation with typed inputs and results	The model, under host control	Something must be calculated, looked up, or changed	<code>get_order_status(order_id)</code>
Resource	Readable context identified by a URI	The host application or user	The model needs reference data such as a document, policy, file, or schema	<code>orders://policy</code>
Prompt	A reusable, optionally parameterized message template	The user or host application	A repeatable workflow should start with consistent instructions	<code>investigate_order(order_id)</code>

Tools

A tool asks the server to **do something**. Its schema tells the client its name, description, arguments, and result shape. A tool may read data or cause side effects, so the host should enforce permissions and approval where necessary. Use a tool for live lookups and actions—not merely to deliver a document.

Resources

A resource lets the client **read something** through a URI. Resources are best for contextual material that can be inspected or attached to a conversation: policies, manuals, configuration, database records, or files. They are commonly treated as read-only context. A resource can be generated dynamically, but its meaning is still data rather than an action.

Prompts

A prompt is a server-provided **message template**. It can accept parameters and return one or more messages that begin a known workflow. Use prompts for repeatable tasks such as reviewing an order, preparing a report, or asking a standard diagnostic sequence. Fetching a prompt does not call a tool or run the model; the client receives messages and decides whether to send them to an LLM.

A practical memory aid:

- **Tool = operation**
- **Resource = context**
- **Prompt = workflow template**

For Lesson 3, the model primarily uses the order **tool**. This lesson also exposes a policy **resource** and an investigation **prompt** so their different roles are concrete.

Setup

Now that the protocol concepts are clear, we can implement them. We use **FastMCP**, a high-level Python framework for building MCP servers and clients. It generates schemas from Python type hints and makes local testing and HTTP deployment concise.

MCP, `mcp`, and FastMCP

These names refer to different layers:

- **MCP** is the open protocol: messages, discovery, capabilities, and transports.
- **`mcp`** is the official Python SDK and reference implementation.
- **FastMCP** is a higher-level framework that implements MCP with additional client, deployment, authentication, composition, and middleware conveniences.

FastMCP still speaks standard MCP. A compatible client does not need to know which Python framework created the server. We choose FastMCP because this course will expose the server as an HTTP service in Lesson 3.

```
In [1]: # Run once if needed
%pip install -qU fastmcp==4.0.5 mcp==2.2.0 pydantic==2.13.5
```

Note: you may need to restart the kernel to use updated packages.

Restart after installation

If this cell installed or upgraded FastMCP, restart the notebook kernel before continuing, then run the cells again from here. Python can keep an older MCP `Tool` class in memory even after `%pip` installs the newer package.

Use **Kernel → Restart Kernel**. The version check below should show FastMCP `4.0.5`, MCP `2.2.0`, and an `output_schema` field.

```
In [2]: from importlib.metadata import version
from mcp.types import Tool

print("FastMCP:", version("fastmcp"))
print("MCP:", version("mcp"))
print("Tool fields:", list(Tool.model_fields))
```

```
if "output_schema" not in Tool.model_fields:
    raise RuntimeError("Restart the kernel, then rerun the notebook from the
```

FastMCP: 4.0.5

MCP: 2.2.0

Tool fields: ['name', 'title', 'description', 'input_schema', 'execution', 'output_schema', 'icons', 'annotations', 'meta']

3. Build a custom order MCP server

The server owns the trusted order data and its contract. The future LLM application will be only a consumer.

```
In [3]: from typing import Literal

from fastmcp import Client, FastMCP
from pydantic import BaseModel, Field

ORDER_DATABASE = {
    "A100": "shipped",
    "B200": "processing",
}

class OrderStatus(BaseModel):
    order_id: str = Field(description="Order identifier supplied by the user")
    status: Literal["shipped", "processing", "not_found"]

order_mcp = FastMCP(
    "Order Service",
    instructions="Use get_order_status for factual order-status lookups.",
)

@order_mcp.tool
def get_order_status(order_id: str) -> OrderStatus:
    """Return the trusted status for one order."""
    status = ORDER_DATABASE.get(order_id, "not_found")
    return OrderStatus(order_id=order_id, status=status)

@order_mcp.resource("orders://policy")
def order_policy() -> str:
    """Explain the meanings of order statuses."""
    return (
        "shipped: handed to carrier; "
        "processing: being prepared; "
        "not_found: no matching order"
    )

@order_mcp.prompt
```

```
def investigate_order(order_id: str) -> str:
    """Create a reusable request for an order investigation."""
    return (
        f"Check order {order_id} with get_order_status, then explain "
        "the result clearly without inventing missing details."
    )
```

What the SDK generated

From the decorated functions, names, docstrings, parameters, and return annotations, FastMCP registers discoverable tools, resources, and prompts. For the tool it also creates:

- tool metadata for discovery;
- JSON input and output schemas;
- argument and result validation;
- protocol request/response handling.

The business logic remains ordinary Python.

4. Connect as a client

For a notebook test, the client connects directly to the server object. Production hosts can use the same client operations over `stdio` or Streamable HTTP.

```
In [4]: async with Client(order_mcp) as client:
    discovered_tools = await client.list_tools()
    discovered_resources = await client.list_resources()
    discovered_prompts = await client.list_prompts()

    for available_tool in discovered_tools:
        print(available_tool.name)
        print(available_tool.description)
        # MCP uses camelCase on the wire. model_dump(by_alias=True) also works
        # with SDK versions whose Python attribute names differ.
        tool_data = available_tool.model_dump(by_alias=True)
        print("Input schema:", tool_data.get("inputSchema"))
        print("Output schema:", tool_data.get("outputSchema"))

    print("Resources:", [str(item.uri) for item in discovered_resources])
    print("Prompts:", [item.name for item in discovered_prompts])
```

get_order_status

Return the trusted status for one order.

Input schema: {'type': 'object', 'additionalProperties': False, 'properties': {'order_id': {'type': 'string'}}, 'required': ['order_id']}

Output schema: {'properties': {'order_id': {'description': 'Order identifier supplied by the user', 'type': 'string'}, 'status': {'enum': ['shipped', 'processing', 'not_found'], 'type': 'string'}}, 'required': ['order_id', 'status'], 'type': 'object'}

Resources: ['orders://policy']

Prompts: ['investigate_order']

The client did not import or inspect the decorated functions. It discovered each capability through MCP. A host can give tool schemas to an LLM, offer prompts in a workflow menu, and choose resources to add as context.

```
In [5]: async with Client(order_mcp) as client:
        result = await client.call_tool(
            "get_order_status",
            {"order_id": "A100"},
            raise_on_error=False,
        )

        print("For the model:", result.content)
        print("Hydrated Python data:", result.data)
        print("Raw structured JSON:", result.structured_content)
        print("Failed:", result.is_error)
```

For the model: [TextContent(type='text', text='{"order_id": "A100", "status": "shipped"}', annotations=None, meta=None)]
Hydrated Python data: Root(order_id='A100', status='shipped')
Raw structured JSON: {'order_id': 'A100', 'status': 'shipped'}
Failed: False

Key idea

One call provides three useful outputs:

- `content` — blocks suitable for the model;
- `data` — FastMCP's hydrated Python result;
- `structured_content` — the standard MCP structured JSON result;
- `is_error` — whether the tool failed.

Always check `is_error` before trusting the structured result.

```
In [6]: async with Client(order_mcp) as client:
        # Resources return readable context.
        policy = await client.read_resource("orders://policy")

        # Prompts return messages; they do not execute the model or a tool.
        investigation = await client.get_prompt(
            "investigate_order",
            {"order_id": "A100"},
        )
```

```
print("Resource:", policy)
print("Prompt messages:", investigation.messages)
```

```
Resource: [TextResourceContents(uri='orders://policy', mime_type='text/plain', meta=None, text='shipped: handed to carrier; processing: being prepared; not_found: no matching order')]
Prompt messages: [PromptMessage(role='user', content=TextContent(type='text', text='Check order A100 with get_order_status, then explain the result clearly without inventing missing details.', annotations=None, meta=None))]
```

5. Run FastMCP as an HTTP service

In-memory transport is best for learning and tests. The same definitions are provided in the companion `order_mcp_server.py` file with this startup block:

```
if __name__ == "__main__":
    order_mcp.run(transport="http", host="127.0.0.1", port=8000)
```

Run it in a terminal:

```
python order_mcp_server.py
```

The MCP endpoint is then `http://127.0.0.1:8000/mcp`. This is MCP over Streamable HTTP—not a conventional REST endpoint. A remote FastMCP client connects with `Client("http://127.0.0.1:8000/mcp")`.

6. Direction for Lesson 3

Lesson 1 currently binds a local LangChain tool. Lesson 3 will keep its prompt, OpenAI model, retries, and `SupportAnswer` schema, but replace the local data function with this server:

Lesson 1 question

- OpenAI selects `get_order_status`
- LangChain/MCP adapter calls Order Service MCP
- FastMCP returns validated data over MCP
- OpenAI produces the grounded `SupportAnswer`

This boundary is intentional: **Lesson 1 owns conversation logic; Lesson 2 owns order capabilities; Lesson 3 connects them.**

The server will not need to be rewritten. Only the host-side adapter and transport configuration are added.

7. Production rules to remember

- Expose narrow, clearly named capabilities.
- Treat tool descriptions and schemas as part of the API contract.

- Validate inputs and structured outputs.
- Authenticate and authorize remote servers.
- Require confirmation for destructive or expensive actions.
- Do not assume an MCP server is trusted merely because it speaks MCP.

Use a direct tool when one application owns the integration. Use MCP when the capability should cross an application, process, framework, or language boundary.

8. Complete code overview

This final cell contains the complete runnable server and in-memory client demo.

```
In [7]: from typing import Literal

from fastmcp import Client, FastMCP
from pydantic import BaseModel, Field

# 1. Trusted data and output contract
ORDER_DATABASE = {"A100": "shipped", "B200": "processing"}

class OrderStatus(BaseModel):
    order_id: str = Field(description="Order identifier supplied by the user")
    status: Literal["shipped", "processing", "not_found"]

# 2. MCP server and capabilities
order_mcp = FastMCP(
    "Order Service",
    instructions="Use get_order_status for factual order-status lookups.",
)

@order_mcp.tool
def get_order_status(order_id: str) -> OrderStatus:
    """Return the trusted status for one order."""
    status = ORDER_DATABASE.get(order_id, "not_found")
    return OrderStatus(order_id=order_id, status=status)

@order_mcp.resource("orders://policy")
def order_policy() -> str:
    """Explain the meanings of order statuses."""
    return (
        "shipped: handed to carrier; "
        "processing: being prepared; "
        "not_found: no matching order"
    )
```

```

@order_mcp.prompt
def investigate_order(order_id: str) -> str:
    """Create a reusable request for an order investigation."""
    return (
        f"Check order {order_id} with get_order_status, then explain "
        "the result clearly without inventing missing details."
    )

# 3. Client: discover, call, and read
async def demo() -> None:
    async with Client(order_mcp) as client:
        tools = await client.list_tools()
        resources = await client.list_resources()
        prompts = await client.list_prompts()
        print("Tools:", [tool.name for tool in tools])
        print("Resources:", [str(item.uri) for item in resources])
        print("Prompts:", [item.name for item in prompts])

        result = await client.call_tool(
            "get_order_status",
            {"order_id": "A100"},
            raise_on_error=False,
        )
        if result.is_error:
            raise RuntimeError(f"MCP tool failed: {result.content}")

        print("Order result:", result.data)

        policy = await client.read_resource("orders://policy")
        investigation = await client.get_prompt(
            "investigate_order", {"order_id": "A100"}
        )
        print("Policy:", policy)
        print("Prompt messages:", investigation.messages)

    await demo()

# When this server is moved to order_mcp_server.py, expose it over HTTP with
# order_mcp.run(transport="http", host="127.0.0.1", port=8000)
# Lesson 3 will connect with Client("http://127.0.0.1:8000/mcp").

```

```

Tools: ['get_order_status']
Resources: ['orders://policy']
Prompts: ['investigate_order']
Order result: Root(order_id='A100', status='shipped')
Policy: [TextResourceContents(uri='orders://policy', mime_type='text/plain',
meta=None, text='shipped: handed to carrier; processing: being prepared; not
_found: no matching order')]
Prompt messages: [PromptMessage(role='user', content=TextContent(type='tex
t', text='Check order A100 with get_order_status, then explain the result cl
early without inventing missing details.', annotations=None, meta=None))]

```

9. Learning checkpoint

You are ready for Lesson 3 if you can explain:

1. Why an MCP tool is still a tool.
2. Why the client discovers schemas instead of importing server functions.
3. When to expose a capability as a tool, resource, or prompt.
4. The roles of host, client, and server.
5. When a direct tool is simpler than MCP.
6. How Lesson 1 will replace its local order lookup with this server.

Small exercise: add a typed `get_order_eta(order_id)` tool. If the client discovers and calls it without client-side schema code, you have built a custom MCP capability.

References

- [FastMCP documentation](#)
- [Official MCP Python SDK](#)
- [MCP architecture](#)
- [MCP server primitives](#)